Entrega 26.2.1 · 9 de noviembre de 2026

SecureUP

Aplicación Android de seguridad digital personal.
Verificación de contraseñas filtradas, escaneo de QR y análisis de URLs.

> **Proyecto:** Desarrollo de Aplicaciones Móviles (binas)
>
> **Stack:** Kotlin · Jetpack Compose · MVVM · Room · Retrofit · CameraX · ML Kit
>
> **Repositorio:** github.com/RyuheiRG/SecureUP (rama `develop`)
>
> **Versión:** 26.2.1 (versionCode 2) · `minSdk 26` · `targetSdk 35`
>
> **Fecha:** 9 de noviembre de 2026

Contenido

1. Resumen ejecutivo
2. Introducción: problema y objetivos
3. Alcance de la entrega y desviaciones
4. Arquitectura MVVM
5. Funcionalidades y flujos
6. Integración con la API de brechas (HIBP)
7. Persistencia con Room
8. Integraciones del dispositivo
9. Seguridad y privacidad
10. Pruebas y calidad
11. Evidencias de funcionamiento
12. Limitaciones y trabajo futuro
13. Conclusiones
14. Referencias

1. Resumen ejecutivo

SecureUP es una aplicación Android que reúne tres herramientas de seguridad personal en una sola cuenta: la verificación de contraseñas expuestas en brechas conocidas, el escaneo de códigos QR con análisis de la URL de destino y el historial local de los escaneos realizados.

La entrega **26.2.1** implementa un corte vertical completo y verificable: autenticación simulada, verificador de brechas contra la API real de Have I Been Pwned, escáner QR con CameraX y ML Kit, análisis heurístico local, persistencia en Room, exportación del historial a PDF y perfil con cierre de sesión. La aplicación compila, pasa el análisis estático y las 67 pruebas unitarias, y fue probada en un dispositivo físico y en un emulador.

67	0	3	52,3 M
pruebas unitarias, 0 fallos	errores de lint	integraciones reales	apariciones de “password” en HIBP

2. Introducción: problema y objetivos

2.1 Problema

Una persona reutiliza contraseñas y abre enlaces sin comprobar su origen. Ambas conductas son frecuentes y difíciles de detectar a simple vista: una contraseña filtrada sigue funcionando hasta que alguien la usa, y un QR puede esconder una URL maliciosa detrás de una apariencia legítima.

Las soluciones habituales exigen enviar la contraseña a un servidor o instalar varias aplicaciones. SecureUP propone lo contrario: verificar sin revelar y analizar antes de abrir, manteniendo los datos en el dispositivo.

2.2 Objetivos

- **General:** construir una app Android con arquitectura MVVM que integre API real, persistencia local e integraciones del dispositivo en un flujo coherente.
- **Específico 1:** verificar contraseñas contra HIBP sin que la contraseña ni su hash completo salgan del teléfono.
- **Específico 2:** leer QR con la cámara o desde una imagen y clasificar la URL antes de permitir abrirla.
- **Específico 3:** persistir el historial en Room y exportarlo a PDF.
- **Específico 4:** demostrar la separación de capas con ViewModels, estados y repositorios, y respaldarla con pruebas.

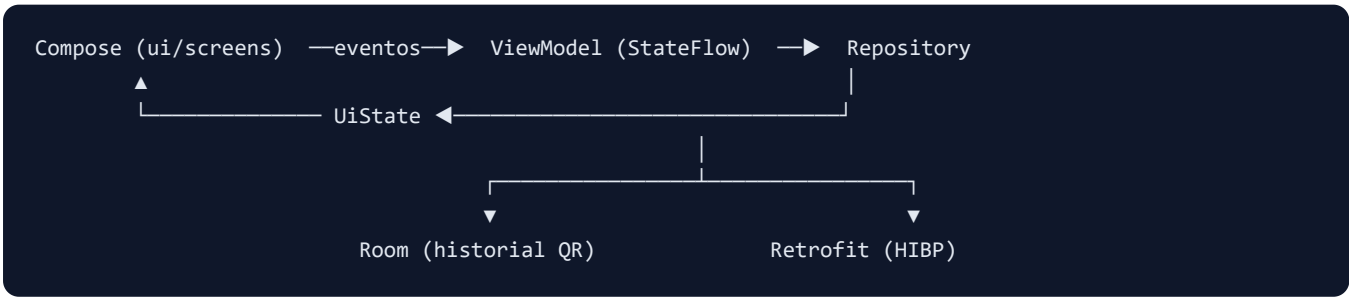
3. Alcance de la entrega y desviaciones

Requisito	Estado	Cómo se cumple
API real	✅	HIBP Pwned Passwords (consulta HTTPS con k-anonimato)
Persistencia	✅	Room 2.8.5, entidad, DAO y esquema exportado
Integración 1	✅	CameraX para la vista de cámara
Integración 2	✅	ML Kit Barcode Scanning (cámara y galería)
Integración 3	✅	SAF + `PdfDocument` y Photo Picker del sistema
Arquitectura MVVM	✅	Pantalla → ViewModel → Repository → Room/Retrofit
Bóveda de contraseñas	⏳	Pospuesta a 26.3.1 por decisión de alcance
Backend propio	⏳	Proyecto separado, aún no disponible

> **Desviaciones declaradas.** (1) La pestaña inferior «Bóveda» se sustituyó temporalmente por «Brechas», porque la bóveda no forma parte de esta entrega. (2) El Security Score se calcula solo con el historial de escaneos: cada URL peligrosa resta 30 puntos y cada sospechosa 10. (3) La verificación de URLs es heurística local; Safe Browsing necesita un proxy en el backend y queda pendiente. (4) El login usa un repositorio simulado en memoria hasta que exista la API propia.

4. Arquitectura MVVM

La aplicación sigue el patrón **Model-View-ViewModel** con una separación estricta de responsabilidades. Las pantallas Compose no contienen lógica de negocio, red, criptografía ni acceso a datos: solo pintan un estado inmutable y emiten eventos.



4.1 Capas y paquetes

Capa	Paquete	Responsabilidad
UI	`ui/screens`, `ui/components`, `ui/navigation`, `ui/theme`	Pintar estado, emitir eventos y navegar. Sin lógica de negocio.
Presentación	`viewmodel`	Exponer un `UiState` por pantalla como `StateFlow` y lanzar operaciones en `viewModelScope`.
Dominio	`domain/model`, `domain/repository`, `domain/security`	Modelos, contratos de repositorio y reglas puras como `UrlAnalyzer`.
Datos	`data/local`, `data/remote`, `data/repository`, `data/scanner`, `data/export`	Implementaciones concretas: Room, Retrofit, ML Kit, PDF.
Composición	`di`	`AppGraph` y `AppContainer`: un service locator simple inicializado en `SecureUpApplication`.

4.2 Decisiones de diseño

- **Un ViewModel por pantalla** con su propio `data class` de estado. La UI nunca muta el estado directamente.
- **Repositorios como contratos** en `domain/repository`. Las pruebas sustituyen HIBP por un doble sin necesidad de red.
- **Operaciones de E/S en `Dispatchers.IO`** y análisis de URL en `Dispatchers.Default`; nunca en el hilo principal.
- **Estado que sobrevive a la navegación** con `collectAsState()` y `StateFlow`; el escáner limpia la URL detectada después de navegar para no repetir la acción al recomponer.
- **Sin dependencias nuevas** fuera de las aprobadas: Room, KSP, Retrofit, CameraX y ML Kit.

5. Funcionalidades y flujos

5.1 Autenticación

Registro y login validan correo y contraseña maestra (mínimo 12 caracteres, confirmación en el registro). Al autenticarse, `AuthViewModel` guarda el `User` en `SessionManager` y la navegación limpia la pila para que «atrás» salga de la app. El cierre de sesión borra la sesión en memoria y vuelve al login.

Limitación: `FakeAuthRepository` acepta cualquier correo válido con contraseña de 12+ caracteres. No hay backend todavía.

5.2 Verificador de brechas

1. El usuario escribe la contraseña. El ViewModel la mantiene solo en memoria.
2. `HibpBreachRepository` normaliza a NFC, calcula SHA-1 y separa prefijo (5 hex) y sufijo (35 hex).
3. Solo el prefijo viaja a `api.pwnedpasswords.com/range/{prefijo}` con la cabecera `Add-Padding: true`.
4. La respuesta se compara localmente; las filas de relleno con contador 0 se ignoran.
5. El ViewModel borra la contraseña del estado al terminar, haya resultado o error.

Resultado real durante la prueba: la contraseña `password` apareció **52 372 427 veces**. La app mostró el badge «Expuesta» y el número formateado.

5.3 Escáner QR y análisis

- **Cámara:** CameraX con `Preview` e `ImageAnalysis` en modo `KEEP_ONLY_LATEST`. Un `AtomicBoolean` evita analizar dos fotogramas a la vez y cada `ImageProxy` se cierra en `addOnCompleteListener`, incluso si ML Kit falla.
- **Galería:** Photo Picker (`PickVisualMedia`), sin permiso de almacenamiento. `QrCodeReader` reduce la imagen antes de decodificarla y recicla el bitmap.
- **Permiso:** `CAMERA` se pide en tiempo de ejecución con explicación; si se deniega, queda la galería.
- **Linterna:** botón superpuesto que alterna `enableTorch`.

5.4 Reglas de `UrlAnalyzer`

Señal	Veredicto
Esquema distinto de `http` / `https`, URL inválida o host vacío	Peligroso
Usuario/contraseña embebidos (`https://banco.com@malicioso.example`)	Peligroso
Sin TLS (`http`), IP literal, punycode (`xn--`), acortador conocido o URL de más de 150 caracteres	Sospechoso
Sin señales	Seguro (sin amenazas conocidas)

Solo las URLs seguras muestran «Abrir en el navegador». Las demás muestran las señales detectadas y un aviso; la URL completa siempre es visible y seleccionable. Nunca se abre automáticamente.

5.5 Historial y PDF

El escaneo se guarda en Room con URL, veredicto y marca de tiempo. La pantalla de historial observa la base de datos y permite:

- **Exportar PDF:** `PdfDocument` genera una tabla con fecha, veredicto y URL (15 filas por página) y la escribe en el `Uri` que devuelve el selector de documentos (SAF). No requiere permisos de almacenamiento.
- **Borrar todo:** diálogo de confirmación y `DELETE` sobre la tabla.
- **Abrir un registro:** vuelve a la pantalla de resultado con el análisis de esa URL.

5.6 Inicio y perfil

`HomeViewModel` observa el historial y calcula el score con datos reales: `100 − 30 × peligrosas − 10 × sospechosas`, acotado entre 0 y 100. También muestra el total de escaneos, las alertas y el último escaneo. El perfil muestra el correo de la sesión, la versión y el cierre de sesión.

6. Integración con la API de brechas (HIBP)

Se usa la API v3 de Pwned Passwords, que no requiere clave. El contrato se declara con Retrofit:

```
@GET("range/{prefix}")
@Headers("Add-Padding: true", "User-Agent: SecureUP-Android")
suspend fun getRange(@Path("prefix") prefix: String): Response<String>
```

Aspecto	Implementación

Privacidad	Solo viaja el prefijo SHA-1 de 5 caracteres; el resto se compara en el dispositivo.
Normalización	`java.text.Normalizer` NFC antes de calcular el hash.
Relleno	`Add-Padding` evita que el tamaño de la respuesta delate el prefijo.
Errores	5xx → servidor; 4xx y `IOException` → red; cuerpo vacío o línea mal formada → respuesta inválida.
Pruebas	7 pruebas con una implementación falsa de `HibpApi` : prefijo enviado, sufijo encontrado, relleno, error de red y NFC.

La API propia (registro, login y vault) sigue pendiente. Cuando exista, el cliente enviará `auth_hash` derivado con PBKDF2 + HKDF y el blob cifrado con AES-256-GCM; la contraseña maestra nunca se enviará.

7. Persistencia con Room

Elemento	Detalle
Entidad	`ScanHistoryEntity` → tabla `scan_history` : `id` autogenerado, `url` , `verdict` (texto), `scannedAt` .
DAO	`observeAll(): Flow<List<ScanRecord>>` , `insert` , `clear` .
Base	`SecureUpDatabase` , versión 1, instancia única con doble comprobación.
Esquema	Exportado en `app/schemas/` y versionado en el repositorio para futuras migraciones.
Repositorio	`RoomScanHistoryRepository` mapea entidad ↔ modelo de dominio.

El veredicto se guarda como texto (`SAFE` , `SUSPICIOUS` , `DANGEROUS`) para que la base siga siendo legible si cambia el orden del enum. El historial no se cifra porque solo contiene URLs y veredictos, no credenciales; la bóveda, que sí contiene secretos, tendrá cifrado AES-GCM en la siguiente entrega.

8. Integraciones del dispositivo

Integración	Uso	Cuidado aplicado
CameraX 1.6.2	Vista previa y análisis de fotogramas	Un análisis a la vez; `unbindAll()` antes de enlazar; fotogramas siempre cerrados.
ML Kit Barcode 17.3.0	Detección de QR en cámara y galería	Procesamiento local; bitmap reducido y reciclado; cliente cerrado al salir.
Photo Picker	Elegir una imagen sin permisos de almacenamiento	Solo se accede al `Uri` elegido por el usuario.
SAF + PdfDocument	Exportar el historial	El destino lo elige el usuario; sin `WRITE_EXTERNAL_STORAGE` .

9. Seguridad y privacidad

Control	Implementación verificada

Contraseña maestra	Nunca se persiste ni se envía; solo vive en memoria durante la sesión.
KDF y cifrado (entrega previa)	PBKDF2-HMAC-SHA256 con mínimo 600 000 iteraciones y HKDF-Expand de un bloque para `auth_hash` y clave del vault; AES-256-GCM con nonce de 12 bytes y AAD = UUID.
Brechas	k-anonimato: solo el prefijo SHA-1 de 5 caracteres; respuesta con relleno.
Red	Solo HTTPS; `usesCleartextTraffic="false"` .
Manifest	`allowBackup="false"` , un solo componente exportado (launcher), permisos `INTERNET` y `CAMERA` en tiempo de ejecución.
Registros	Sin `Log.*` en `app/src/main` ; no se registran contraseñas, hashes ni tokens.
QR	URL no confiable: se muestra completa, se analiza antes de abrir y nunca se abre sola; solo `http` / `https` .
Secretos	Ninguna API key en la app ni en el repositorio.

> **Verificación automática:** `lintDebug` terminó con **0 errores** y 25 advertencias (targetSdk no reciente, tamaños de vectores heredados y avisos de versiones). No se encontraron llamadas a `Log` en el código de producción.

10. Pruebas y calidad

10.1 Pruebas unitarias

Clase	Pruebas	Qué cubre
`CryptoManagerTest`	28	Vectores de PBKDF2/HKDF, nonce distinto por operación, manipulación de ciphertext y AAD.
`UrlAnalyzerTest`	11	HTTPS seguro, HTTP, IP, punycode, acortador, user-info, esquemas peligrosos y URL larga.
`HibpBreachRepositoryTest`	7	Prefijo de 5, sufijo encontrado, relleno, error de red, respuesta vacía y NFC.
`AuthValidatorTest`	6	Reglas de correo y contraseña maestra.
`AuthViewModelTest`	5	Estados de carga, error y éxito.
`BreachViewModelTest`	5	Resultado, error, carga y borrado de la contraseña.
`FakeAuthRepositoryTest`	4	Login y registro simulados, incluido el correo de error.
`ExampleUnitTest`	1	Prueba base del proyecto.
Total	**67**	**0 fallos** · `testDebugUnitTest`

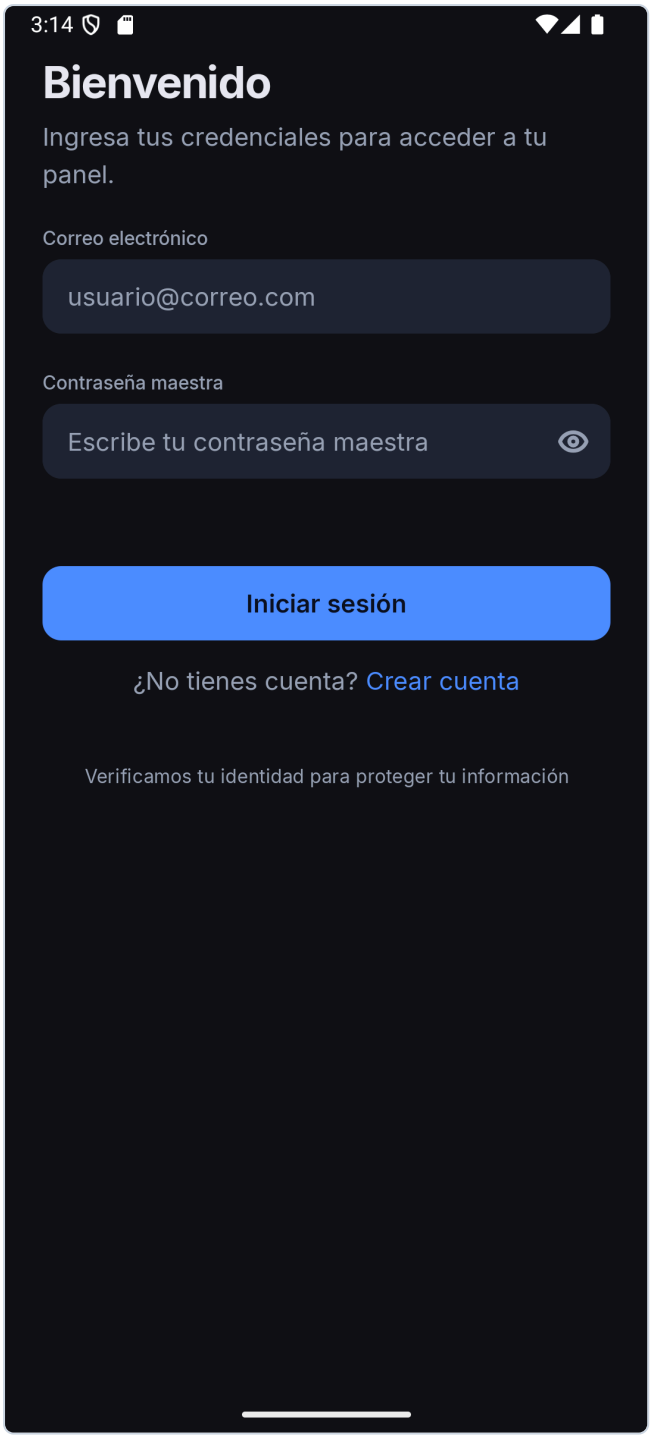
Además hay 6 pruebas instrumentadas de `CryptoManager` que repiten los vectores en el proveedor criptográfico de Android (Conscrypt) y una prueba base de instrumentación.

10.2 Pruebas manuales en dispositivo

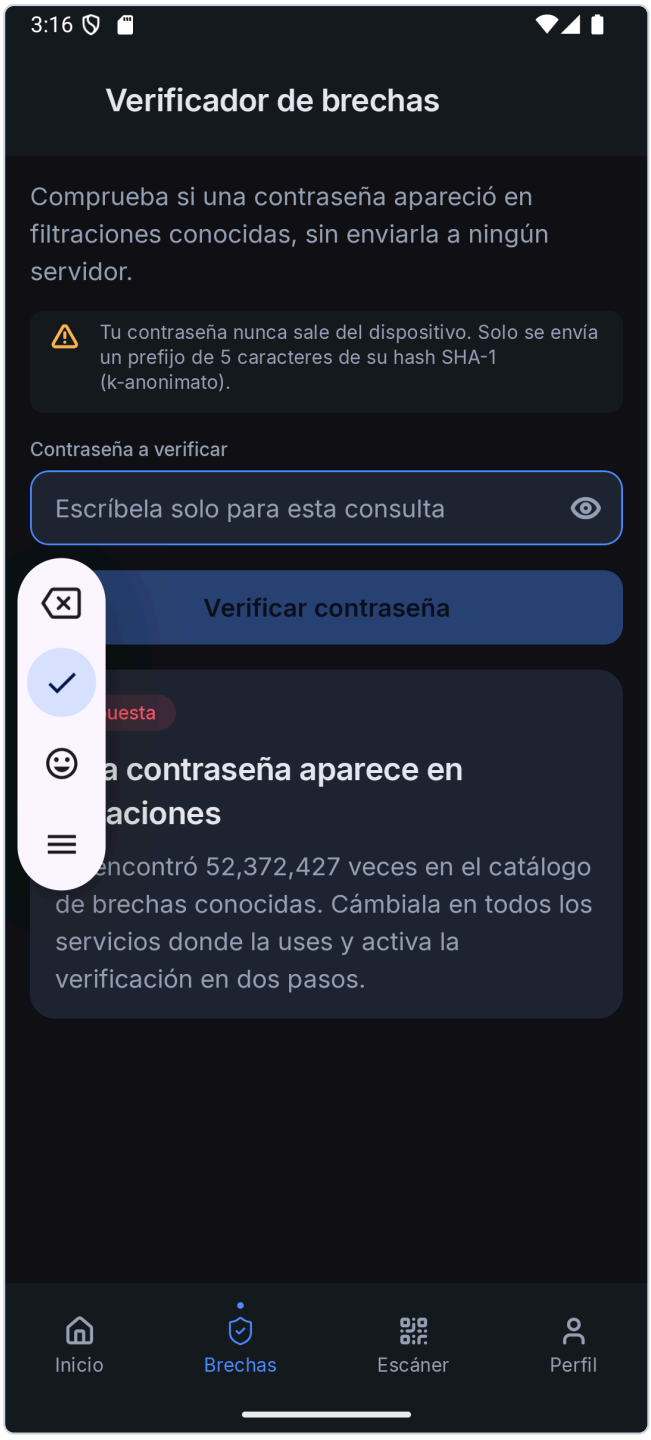
Prueba	Resultado

Instalación del APK debug en teléfono físico (Android 16 / API 36) y emulador (API 35)	Correcta
Onboarding → login con credenciales válidas	Navega al Inicio con sesión
Verificación real de `password` en HIBP	«Expuesta» · 52 372 427 apariciones
QR seguro desde galería (`https://example.com`)	«Seguro» y botón para abrir
QR peligroso (`https://banco.com@malicioso.example/login`)	«Peligroso», señal de user-info y sin botón de abrir
Cerrar el proceso y volver a entrar	El historial, las alertas y el score se conservan (Room)
Exportar historial a PDF	Archivo de 40 160 bytes generado en Descargas
Cerrar sesión	Vuelve al login y limpia la sesión

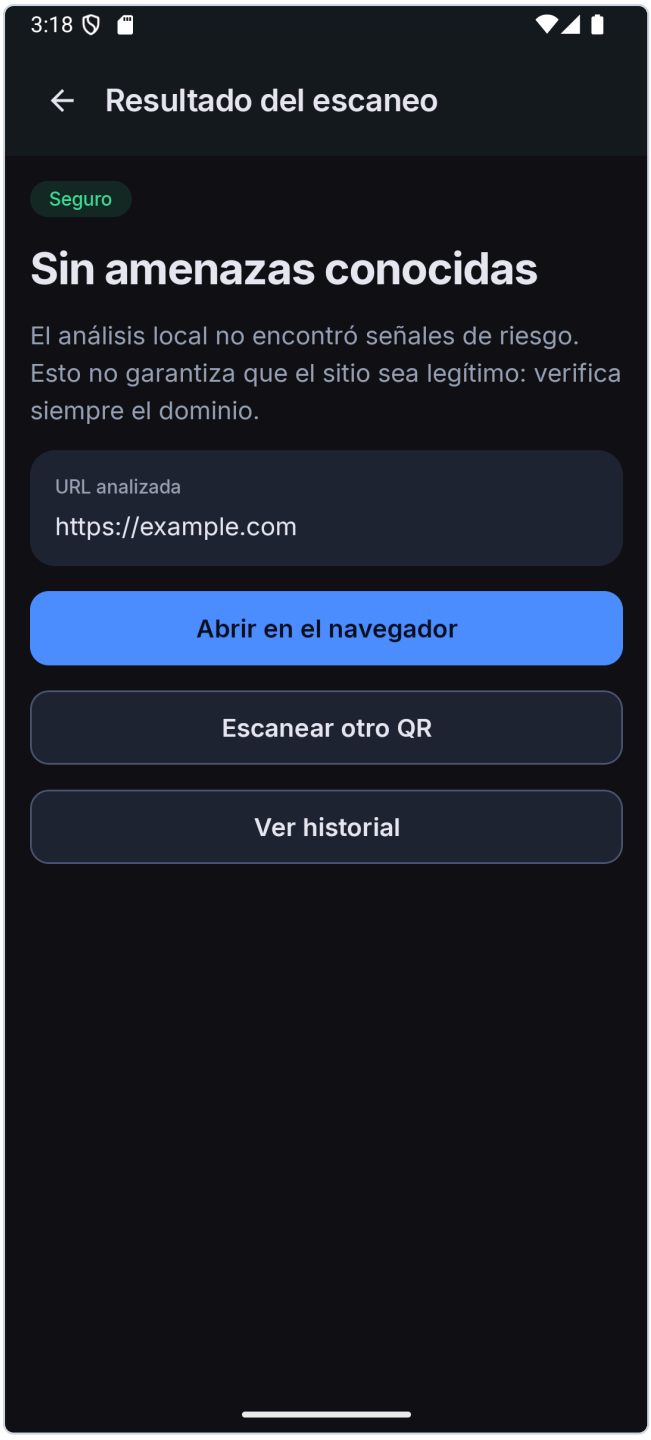
11. Evidencias de funcionamiento



Login con validación



HIBP: «password» expuesta 52 372 427 veces



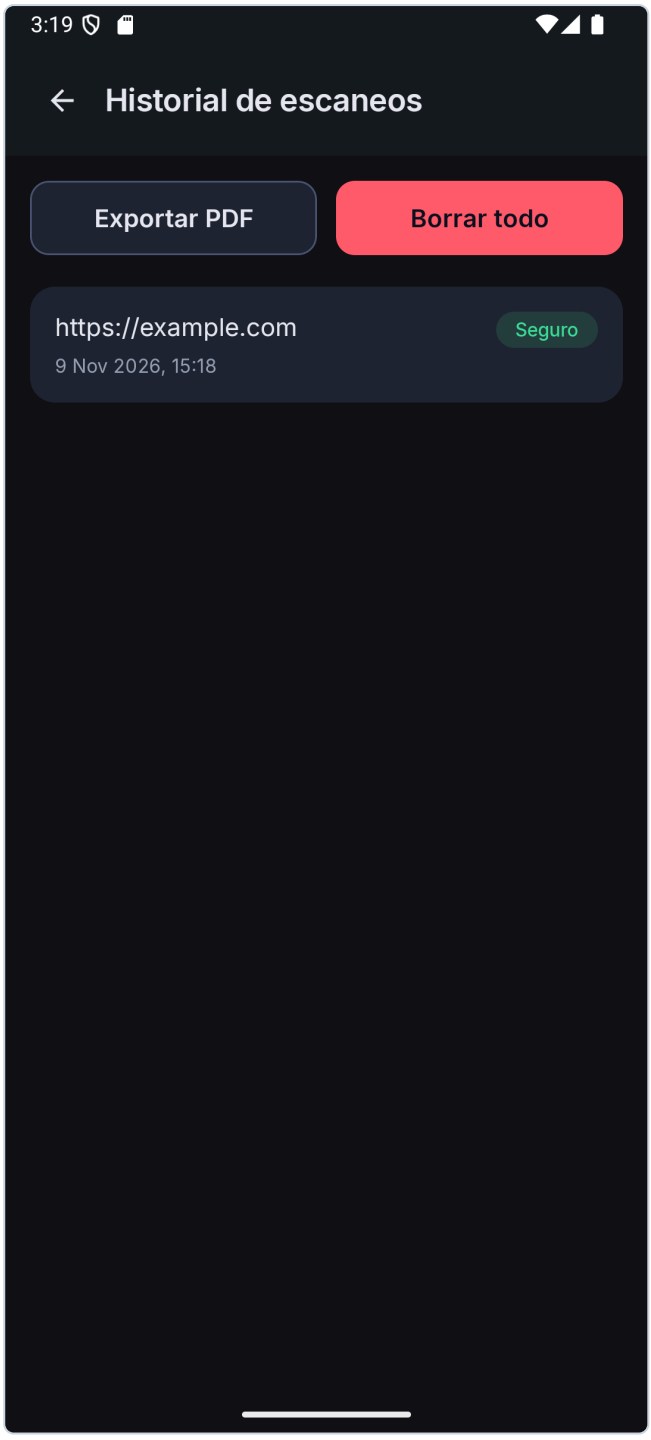
QR seguro desde galería



QR peligroso: no se ofrece abrir



Inicio con score real (70) y última alerta



Historial persistido en Room

Las capturas se tomaron en el emulador API 35 durante la verificación de esta entrega. El PDF de ejemplo está en `docs/evidencias/ejemplo-historial.pdf`.

12. Limitaciones y trabajo futuro

Limitación	Impacto	Trabajo futuro
Sin backend propio	Login y registro son simulados; no hay cuentas reales ni sincronización.	Implementar la API REST (registro, login, vault) y sustituir `FakeAuthRepository`.
Sin bóveda	No se guardan credenciales todavía.	Room cifrado + AES-256-GCM, pantallas de lista, detalle y alta.

Análisis de URL heurístico	No detecta campañas nuevas ni dominios recién registrados.	Proxy de Google Safe Browsing en el backend.
Score parcial	Solo considera escaneos, no la fortaleza de contraseñas.	Incorporar las métricas de la bóveda.
Historial sin cifrar	Contiene URLs visitadas, no secretos.	Valorar cifrado si se añaden datos sensibles.
Cambio de contraseña maestra y eliminación de cuenta	Pendientes de diseño y backend.	Implementarlos junto con la bóveda.

13. Conclusiones

La entrega demuestra un corte vertical funcional: una pantalla Compose que no sabe de red, un ViewModel que orquesta, un repositorio que decide el origen del dato y una fuente real (HIBP o Room). Las tres integraciones —API, cámara/ML y PDF/SAF— funcionan en un dispositivo y están cubiertas por pruebas o por verificación manual documentada.

El resultado más importante es de privacidad: la verificación de brechas se hizo contra el servicio real sin enviar la contraseña ni su hash completo. Ese mismo principio guiará la bóveda de la siguiente entrega: una sola derivación costosa, claves que nunca salen del dispositivo y cifrado autenticado por entrada.

14. Referencias

- Android Developers. *Guide to app architecture*. developer.android.com/topic/architecture
- Android Developers. *Save data in a local database using Room*. developer.android.com/training/data-storage/room
- Android Developers. *CameraX overview*. developer.android.com/media/camera/camerax
- Google ML Kit. *Barcode Scanning*. developers.google.com/ml-kit/vision/barcode-scanning/android
- Have I Been Pwned. *Pwned Passwords API v3*. haveibeenpwned.com/API/v3#PwnedPasswords
- OWASP. *Password Storage Cheat Sheet*. cheatsheetseries.owasp.org
- OWASP. *Mobile Application Security Verification Standard (MASVS)*.
- Square. *Retrofit*. square.github.io/retrofit/